

Comparativo de Implementações em Python

(Análise e Transcrição de Algoritmos)

1. Diferenças Estruturais Gerais

Paradigma de Programação

O **primeiro documento** (Código Melhorado) adota uma arquitetura modular, isolando a lógica de negócio em funções específicas utilizando a instrução `def`. Em contrapartida, o **segundo documento** (Algoritmos Resolvidos) utiliza uma abordagem procedural linear, executando as instruções de forma direta, de cima para baixo (como um script simples).

Tratamento de Exceções e Erros

O primeiro código implementa blocos preventivos `try...except` e levantamento explícito de erros com `raise ValueError` para lidar com problemas matemáticos e entradas mal formatadas. Já o segundo código confia em desvios condicionais simples (`if/else`) para evitar falhas de execução pontuais e exibir avisos no terminal.

2. Comparativo Detalhado por Exercício

Exercício 1: Conversor de Tempo

- **PDF 1:** Possui validação defensiva que impede a execução caso o usuário insira um tempo de entrada negativo (`if tempo < 0`).
-

PDF 2: Não valida valores negativos, mas cria variáveis intermediárias declaradas de forma explícita (`resto1` e `resto2`) para armazenar os restos das sucessivas divisões modulares.

Exercício 2: Ponte de Wheatstone

- **PDF 1:** Para contornar o problema da divisão por zero caso a resistência 1 seja nula, o script interrompe a execução levantando um erro nativo da linguagem (`raise ValueError("R1 não pode ser zero.")`).
- **PDF 2:** Gerencia o mesmo cenário exibindo uma mensagem de texto de erro no terminal e isolando o restante da lógica funcional dentro de um bloco `else` .

Exercício 3: Cálculo Tributário em Cascata

Divergência Matemática Importante: A equação que define a porcentagem final da carga tributária apresenta naturezas distintas:

- **PDF 1:** Calcula o aumento percentual da carga utilizando como base divisora o **preço de fábrica** original: $((\text{preco_final} - \text{preco_fabrica}) / \text{preco_fabrica}) * 100$.
- **PDF 2:** Avalia a representatividade da carga tributária no produto utilizando como divisor o **preço final**: $((\text{preco_final} - \text{preco_fabrica}) / \text{preco_final}) * 100$.

Exercício 4: Laudo de Sondagem de Solo (SPT)

- **PDF 1:** Retorna o diagnóstico da análise do solo em uma string textual única e consolidada, sem detalhar os motivos limitantes de forma separada.
- **PDF 2:** Adiciona fluxos de validação extras, responsáveis por apontar ativamente qual foi a variável limitante que rebaixou a nota do laudo (imprimindo avisos focados, como `Parâmetro Limitante: SPT baixo` ou `Parâmetro Limitante: Carga baixa`).

Exercício 5: Controle de Qualidade e Esteira

- **PDF 1:** A verificação do limite de tolerância da solda usa a função de valor absoluto nativa, checando se o módulo do desvio é menor ou igual à tolerância nominal estipulada: `abs(desvio) <= tolerancia` .
- **PDF 2:** Realiza a checagem prescindindo da função de valor absoluto, adotando uma desigualdade matemática encadeada, muito característica da linguagem Python, que compara limites negativos e positivos simultaneamente: `-tolerancia_abs <= desvio <= tolerancia_abs` .